BOM nodoen zuhaitza

Internet Explorer eta Netscape Navigator nabigatzaileen 3.0 bertsioek Browser Object Model edo BOM kontzeptua sartu zuten, nabigatzailearen leihoen propietateak eskuratu eta aldatzeko aukera ematen duena.

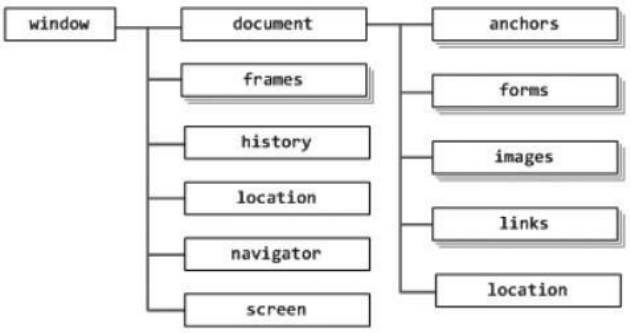
BOM erabiliz, arakatzailearen leihoaren tamaina aldatzea eta mugitzea posible da, egoera-barran bistaratzen den testua aldatzea eta HTML orriaren edukiarekin zerikusirik ez duten beste manipulazio asko egitea.

BOM-en eragozpenik handiena da, DOM-ekin gertatzen dena ez bezala, **ez dagoela** nabigatzaile desberdinen artean **estandarizatzeaz** edo elkarreragingarritasuna bermatzeaz **arduratzen den organismorik**.

BOM osatzen duten elementuetako batzuk hauek dira:

- Sortu, mugitu, tamaina aldatu eta itxi arakatzailearen leihoak.
- Lortu arakatzaileari berari buruzko informazioa.
- Uneko orriaren eta erabiltzailearen pantailaren propietateak.
- Cookieen kudeaketa.
- ActiveX objektuak Internet Explorer-en.

BOM-a elkarrekin erlazionatutako hainbat objektuk osatzen dute. Ondorengo diagramak BOM objektuak eta haien erlazioa erakusten ditu:



Goiko diagraman, gainjarritako hainbat koadroekin erakusten diren objektuak matrizeak dira (frames, anchors, forms eta links). Gainerako objektuak, laukizuzen bakarraz irudikatuak, objektu sinpleak dira. Nolanahi ere, objektu guztiak *window* objektutik eratortzen dira.

Window objektua

Window objektuak arakatzailearen leiho osoa adierazten du. Objektu hau erabiliz, uneko arakatzailearen leihoa mugitu, tamainaz aldatu eta manipulatu daiteke. Are gehiago, arakatzailearen leiho berriak ireki eta ixteko aukera dago.

Beste objektu guztiak leiho-objektutik zuzenean edo zeharka heredatzen direnez, ez dago JavaScript kodean esplizituki adierazi beharrik. Bestela esanda:

```
window.forms[0] === forms[0]
window.document === document
```

BOM-ek lau metodo definitzen ditu leihoaren tamaina eta posizioa manipulatzeko:

- **moveBy(x,y):** leihoaren posizioa *x* pixel eskuinera eta *y* pixel behera mugitzen du. Desplazamendu negatiboek leihoa ezkerrera edo gora mugitzen dute.
- **moveTo(x,y):** arakatzailea pantailarekiko x eta y posiziora mugitzen du, bietan goi-ezkerrko puntua errerferentzia bezela hartuz. Desplazamendu negatiboak onartzen dira, arakatzailea pantailaz kanpu utziz.
- **resizeBy(x,y):** arakatzailearen leihoaren tamaina aldatzen du, bere zabalera berria (aurreko_zabalera + x) eta altuera berria (aurreko_altuera + y) berdina izan dadin. Balio negatiboak leihoaren zabalera eta/edo altuera murrizteko erabil daitezke. **Batura izan daiteke negatiboa?**
- **resizeTo(x,y):** arakatzailearen leihoaren tamaina aldatzen du bere zabalera x eta altuera y berdin arte. Balio negatiboak ez dira onartzen.

Arakatzaileak gero eta baimen gehiago ematen diote JavaScript-i lehioetan aldaketak egiteko. Bestalde, arakatzaile gehienek erabiltzaileei JavaScript-en erabilera blokeatzeko aukera ematen diete mota honetako aldaketak egozteko. Honela, horria ez bada guztiz fidagarria, erabiltzaileak kodea ez exekutatzeko aukera izango luke.

Arakatzailearen leihoa mugitzeaz eta tamainaz aldatzeaz gain, leihoaren uneko posizioa eta tamaina jakin daiteke. Hala ere, BOM estandarrik ez dagoenez arakatzaile bakoitzak bere modura egiten du:

- **Internet Explorer**-ek *window.screenLeft* eta *window.screenTop* propietateak eskaintzen ditu leihoaren kokapen-koordenatuak lortzeko. Ezin da leiho osoaren tamaina lortu, orriaren ikusgai dagoen eremua bakarrik (hau da, egoera-barra edo menurik gabe). Dimentsio hauek ematen dituzten propietateak *document.body.offsetWidth* eta *document.body.offsetHeight* dira.
- **Mozilla, Safari eta Opera** familiako arakatzaileek *window.screenX* eta window.screenY propietateak eskaintzen dituzte leihoaren posizioa lortzeko. Leihoaren eremu ikusgaiaren tamaina *window.innerWidth* eta *window.innerHeight* bidez lortzen da, eta leihoaren guztizko tamaina *window.outerWidth* eta *window.outerHeight* bidez lortzen da.

Document objektua

Document objektua DOM-ari zein BOMri dagokion bakarra da. BOM-aren ikuspuntutik, dokumentu-objektuak HTML orrialdeari berari buruzko informazioa ematen du.

Dokumentu objektuak definitutako propietate garrantzitsuenetako batzuk hauek dira:

Propiedad	Descripción
`lastModified`	La fecha de la última modificación de la página
`referrer`	La URL desde la que se accedió a la página (es decir, la página anterior en el array `history`)
`title`	El texto de la etiqueta `<title>`
`URL`	La URL de la página actual del navegador

Title eta *URL* propietateak irakurri eta idaz daitezke eta beraz, haien balioa lortzeaz gain, zuzenean ezar daitezke:

```
// modificar el título de la página
document.title = "Nuevo titulo";

// llevar al usuario a otra página diferente
document.URL = "http://nueva_pagina";
```

Propietateez gain, *document* objektuak orrialdeko elementu batzuei buruzko informazioa duten hainbat *array* ditu:

Array	Descripción
`anchors`	Contiene todas las "anclas" de la página (los enlaces de tipo `<a name="nombre_ancla"></a>`)
`applets`	Contiene todos los applets de la página
`embeds`	Contiene todos los objetos embebidos en la página mediante la etiqueta `<embed>`
`forms`	Contiene todos los formularios de la página
`images`	Contiene todas las imágenes de la página
`links`	Contiene todos los enlaces de la página (los elementos de tipo `<a href="enlace.html"></a>`)

Document objektuaren array bakoitzeko elementuak zenbakizko indizearen bidez edo HTML orrialdeko elementuaren izenaren bidez eskura daitezke. Hurrengo HTML orrian adibidez:

```
<html>
  <head><title>Pagina de ejemplo</title></head>
  <body>
    <p>Primer parrafo de la pagina</p>
    <a href="otra_pagina.html">Un enlace</a>
    <img src="logo.jpg" name="logotipo"/>
    <form method="post" name="consultas">
      <input type="text" name="id" />
      <input type="submit" value="Enviar">
    </form>
  </body>
</html>
```

Orriaren elementuetara sartzeko DOM funtzioak edo BOM objektuak erabil ditzakezu:

- Parrafoa: *document.getElementsByTagName("p")*
- Esteka: *document.links[0]*
- Irudia: *document.images[0] o document.images["logotipo"]*
- Formularioa: *document.forms[0] o document.forms["consultas"]*

Elementuaren erreferentzia lortu ondoren, bere HTML atributuen balioa DOM propietateak erabiliz atzitu daiteke. Horrela, inprimaki-metodoa *document.forms["queries"].method* bidez lortzen da eta irudiaren *document.images[0].src* bidez.

Location objektua

Location objektua BOM-ko objekturik erabilgarrienetako bat da. Estandarizazio falta dela eta, *location* leiho objektuaren zein *document* objektuaren propietate bat da.

Location objektuak arakatzailearen leihoan bistaratzen den HTML orriaren URLa adierazten du eta URLa kudeatzeko hainbat propietate erabilgarria eskaintzen ditu:

Propiedad	Descripción
`hash`	El contenido de la URL que se encuentra después del signo `#` (para los enlaces de las anclas) `http://www.ejemplo.com/ruta1/ruta2/pagina.html#seccion hash` - `#seccion`
`host`	El nombre del servidor `http://www.ejemplo.com/ruta1/ruta2/pagina.html#seccion host` - `www.ejemplo.com`
`hostname`	La mayoría de las veces coincide con `host`, aunque en ocasiones, se eliminan las `www` del principio `http://www.ejemplo.com/ruta1/ruta2/pagina.html#seccion hostname` - `www.ejemplo.com`
`href`	La URL completa de la página actual `http://www.ejemplo.com/ruta1/ruta2/pagina.html#seccion URL` - `http://www.ejemplo.com/ruta1/ruta2/pagina.html#seccion`
`pathname`	Todo el contenido que se encuentra después del host `http://www.ejemplo.com/ruta1/ruta2 /pagina.html#seccion pathname` - `/ruta1/ruta2/pagina.html`
`port`	Si se especifica en la URL, el puerto accedido `http://www.ejemplo.com:8080/ruta1/ruta2 /pagina.html#seccion port` - `8080` La mayoría de URL no proporcionan un puerto, por lo que su contenido es vacío `http://www.ejemplo.com/ruta1/ruta2/pagina.html#seccion port` - (vacío)
`protocol`	El protocolo empleado por la URL, es decir, todo lo que se encuentra antes de las dos barras inclinadas `// http://www.ejemplo.com/ruta1/ruta2/pagina.html#seccion protocol` - `http:`
`search`	Todo el contenido que se encuentra tras el símbolo `?`, es decir, la consulta o *"query string"* `http://www.ejemplo.com/pagina.php?variable1=valor1&variable2=valor2 search` - `?variable1=valor1&variable2=valor2`

Propietate guztien artean, gehien erabiltzen dena *location.href* da, arakatzailearen leihoan bistaratzen den orriaren helbidea eskuratu edo ezartzeko aukera ematen duena.

Navigator objektua

Navigator objektua BOM-an sartutako lehen objektuetako bat da eta nabigatzaileari berari buruzko informazioa lortzeko aukera ematen du. Internet Explorer-en, *navigator* objektua *clientInformation* objektuaren bidez ere sar daiteke.

Gutxien estandarizatu den objektuetako bat den arren, bere propietate batzuk ohikoak dira ia nabigatzaile guztietan. Hona hemen propietate horietako batzuk:

Propiedad	Descripción
`appCodeName`	Cadena que representa el nombre del navegador (normalmente es `Mozilla`)
`appName`	Cadena que representa el nombre oficial del navegador
`appMinorVersion`	(Sólo Internet Explorer) Cadena que representa información extra sobre la versión del navegador
`appVersion`	Cadena que representa la versión del navegador
`browserLanguage`	Cadena que representa el idioma del navegador
`cookieEnabled`	Boolean que indica si las cookies están habilitadas
`cpuClass`	(Sólo Internet Explorer) Cadena que representa el tipo de CPU del usuario ("x86", "68K", "PPC", "Alpha", "Other")
`javaEnabled`	Boolean que indica si Java está habilitado
`language`	Cadena que representa el idioma del navegador
`mimeTypes`	Array de los tipos MIME registrados por el navegador
`onLine`	(Sólo Internet Explorer) Boolean que indica si el navegador está conectado a Internet
`oscpu`	(Sólo Firefox) Cadena que representa el sistema operativo o la CPU
`platform`	Cadena que representa la plataforma sobre la que se ejecuta el navegador
`plugins`	Array con la lista de plugins instalados en el navegador
`preference()`	(Sólo Firefox) Método empleado para establecer preferencias en el navegador
`product`	Cadena que representa el nombre del producto (normalmente, es `Gecko`)
`productSub`	Cadena que representa información adicional sobre el producto (normalmente, la versión del motor Gecko)
`securityPolicy`	Sólo Firefox
`systemLanguage`	(Sólo Internet Explorer) Cadena que representa el idioma del sistema operativo
`userAgent`	Cadena que representa la cadena que el navegador emplea para identificarse en los servidores
`userLanguage`	(Sólo Explorer) Cadena que representa el idioma del sistema operativo
`userProfile`	(Sólo Explorer) Objeto que permite acceder al perfil del usuario

Navigator objektua nabigatzaile mota eta/edo bertsioa detektatzeko erabiltzen da nabigatzaile bakoitzeko kodea desberdina duten aplikazioetan. Horrez gain, nabigatzaileak cookieak eta Java gaituta dituen detektatzeko eta arakatzailean dauden pluginak egiaztatzeko ere balio du.

Screen objektua

Screen objektua erabiltzailearen pantailari buruzko informazioa lortzeko erabiltzen da. Pantailaren objektuak ematen duen datu garrantzitsuenetako bat orriak bistaratzen ari diren monitorearen bereizmena (resolution) da. Propietate hauek eskuragarri daude *screen* objektuan:

Propiedad	Descripción
`availHeight`	Altura de pantalla disponible para las ventanas
`availWidth`	Anchura de pantalla disponible para las ventanas
`colorDepth`	Profundidad de color de la pantalla (32 bits normalmente)
`height`	Altura total de la pantalla en píxel
`width`	Anchura total de la pantalla en píxel

Pantailaren altuera/zabalera eskuragarria pantailaren altuera/zabalera osoa baino txikiagoa da, sistema eragilearen elementuen tamaina kontuan hartuta, hala nola ataza-barra eta leihoen arakatzailea.

Erabiltzaileen ordenagailuetan estatistikak biltzeaz gain, pantaila-objektuaren propietateak erabiltzen dira, adibidez, leiho bat nola eta zenbateraino alda daitekeen zehazteko eta erabiltzailearen pantailan zentratuta dagoen leiho bat jartzeko.

Hurrengo adibidean leiho berri bat tamainaz aldatzen da ahalik eta gehieneko tamainara erabiltzailearen pantailan oinarrituta:

```
window.moveTo(0, 0);
window.resizeTo(screen.availWidth, screen.availHeight);
```

History objektua

Objektu hau arakatu diren guneen zerrenda gordetzeaz arduratzen da, hau da, bisitatutako lekuen erreferentziak gordetzen ditu. Zerrenda horretan aurrera edo atzera egiteko erabiltzen da, batez ere. Hauek dira bere propietateak:

- **Current**: Uneko historiako sarreraren URL osoa duen katea.
- **Next:** Hurrengo historiako sarreraren URL osoa duen katea.
- **Length**: Historiako sarrera kopurua (hau da, zenbat helbide bisitatu diren) duen zenbaki osoa.
- **Previous**: Aurreko historiako sarreraren URL osoa duen katea.

Eta hauek dira objektu honen metodo batzuk:

- **Back():** Historiaren barruan aurreko dokumentuaren URLa birkargatzen du.
- **Forward():** Historiako hurrengo dokumentuaren URLa berriro kargatzen du.
- **Go(posizioa):** Historiako posizioaren arabera zehaztutako dokumentuaren URLa birkargatzen du. Posizioa zenbaki oso bat izan daiteke, eta kasu horretan dokumentuaren posizio erlatiboa historiaren barruan adierazten du; edo karaktere-kate bat izan daiteke, eta kasu horretan historian dagoen URL osoa edo zati bat adierazten du.

Denbora kudeatzeko

Beste programazio-lengoaia batzuetan ez bezala, JavaScript-ek ez du *wait()* metodorik programak denbora jakin batean exekutatzen uzten duen metodorik sartzen. Hala ere, JavaScript-ek *setTimeout()* eta *setInterval()* metodoak eskaintzen ditu, antzeko zereginak egiteko erabil daitezkeenak.

SetTimeout() metodoak denbora-tarte jakin bat igaro ondoren funtzio bat exekutatzeko aukera ematen du. *setTimeout()* metodoak bi argumentu behar ditu. Lehenengoa exekutatu beharreko kodea edo exekutatu beharreko funtzioaren erreferentzia da. Bigarren argumentua kodea exekutatzen hasi arte itxaron beharreko denbora da, milisegundotan.

```
function muestraMensaje() {
  console.log("Han transcurrido 3 segundos desde que me programaron");
}

setTimeout(muestraMensaje, 3000);
```

Atzerako kontaketa ezartzen denean, *setTimeout()* funtzioak atzerako kontaketa berri horren identifikatzailea itzultzen du. Identifikatzaile hau eta *clearTimeout()* funtzioa erabiliz, zain dagoen kodea exekutazea sahiestea ahalbidetzen du:

```
function muestraMensaje() {
  console.log("Han transcurrido 3 segundos desde que me programaron");
}
var id = setTimeout(muestraMensaje, 3000);

// Antes de que transcurran 3 segundos, se decide eliminar la ejecución pen
diente
clearTimeout(id);
```

Funtzio baten etorkizuneko exekuzioa programatzeaz gain, JavaScript-ek funtzio baten exekuzio aldizkako eta errepikakorra ezartzeko aukera ematen du. Beharrezko metodoa *setInterval()* da eta bere funtzionamendua *setTimeout*()-en berdina da.

*ClearTimeout(*)-en antzera, aldizkako errepikapen bat ezabatzeko aukera ematen duen metodo bat ere badago eta kasu honetan *clearInterval()* deitzen da.

```
function muestraMensaje() {
  console.log("Este mensaje se muestra cada segundo");
}

var id = setInterval(muestraMensaje, 1000);

// Despues de ejecutarse un determinado número de veces, se elimina el inte
rvalo
clearInterval(id);
```